

Capítulo 1

PWN College - Computer 101

Estructura y Contenido del Capítulo

1.1. The Stack, Revisited	1
---------------------------	---

1.1. The Stack, Revisited

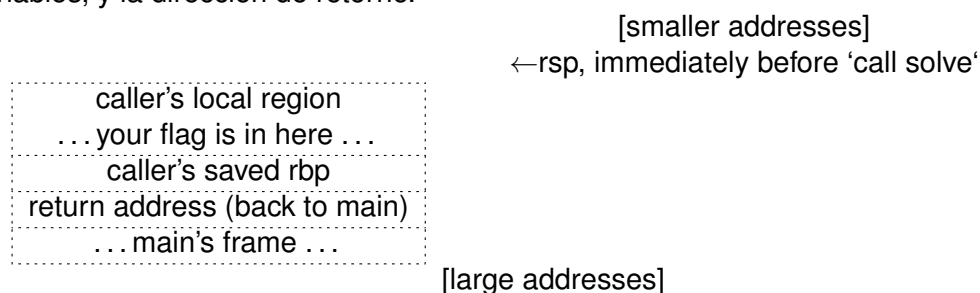
Subsecciones del Tema

- › Reaching Into the Caller's Frame
- › Loading Stale Stack Data
- › Stealing Stale Stack Data
- › Reserving Your Own Frame
- › Using Your Own Frame
- › Environment Variables on the Stack
- › Aligning the Stack Through the Environment
- › Aligning the Stack Through GDB
- › Aligning the Stack Through GDB, Generalized

1.1.1. Reaching Into the Caller's Frame

La pila almacena las variables locales de las funciones: datos que estas utilizan para funciones que no son necesariamente necesarias para otras funciones del programa.

Por lo tanto, tú podrías acceder al marco de una función para poder conocer los valores de sus variables, y la dirección de retorno.



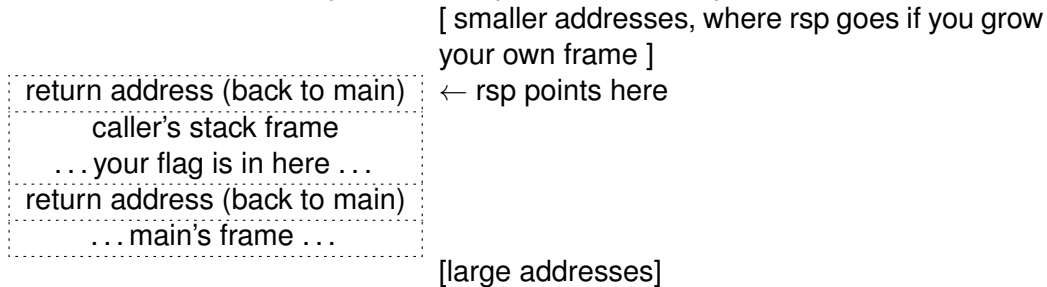
La instrucción `call solve` realiza 2 acciones:

1. Empula la dirección de retorno a la pila (8bytes), por lo que la dirección de retorno acaba ocupando una dirección menor que la que ya había en la pantalla.
2. Salta a tu código.

El primer paso es fundamental, puesto que la pila crece al revéz. La instrucción `pop` suma 8 a `rsp` y la instrucción `push` resta 8, equivalencias:

- `pop rdi = mov rdi, [rsp] y add rsp, 8`
- `push rdi = sub rsp, 8 y mov [rsp], rdi`

instrucción Al momento de ejecutarse, la pila tiene este aspecto:



Flag Capturada

✓ CAPTURADA

pwn.college{wtu3_01ESb6TzTa1PtIXgZ9ZjUS.0F01kDNywSMycjN4EzW}

1.1.2. Loading Stale Stack Data

En este caso, vamos a utilizar los bytes, que se dejaron atrás cuando una función ya devolvió su resultado.

Cuando una función termina y regresa, los bytes que utilizo no se eliminan, sino que se quedan obsoletos, esto por un tema de eficiencia.

Cuando los datos que quedan son confidenciales, omitir ese borrado puede convertirse en una vulnerabilidad.

Flag Capturada

✓ CAPTURADA

pwn.college{AZKP7tCbsW41IyFS_Zzsf4hgHU2.01N2ITNywSMycjN4EzW}

1.1.3. Stealing Stale Stack Data

Se utilizará la misma idea de los datos residuales en la pila con un búfer de bytes.

En este reto, nos pide que llamemos a la función `read_flag`, y luego imprimir por **stdout** los bytes obsoletos de la flag provenientes del marco de la función llamada.

Flag Capturada

✓ CAPTURADA

pwn.college{EVdj0B2CbkZqndmuAe6SpseIdZF.01NzITNywSMycjN4EzW}

1.1.4. Reserving Your Own Frame

Hasta el momento, las funciones mantienen sus valores temporales en los registros; pero una función puede necesitar más espacio de trabajo (*scratch space*) del que pueden contener los registros.

En la arquitectura x86, una función crea espacio de trabajo en la pila modificando el puntero de la pila `rsp`, para que apunte a una dirección más baja (`sub rsp, 256`).

Solo se tiene que considerar 3 cosas:

1. Ahora se van a necesitar diferentes desplazamientos(*offset*) relativos a `rsp`.
2. Los bytes que se acaban de mover, son memoria común de la pila, y pueden contener datos dejados por código ejecutado anteriormente.
3. Se tiene que devolver `rsp` a su posición original antes de retornar.

Flag Capturada

✓ CAPTURADA

`pwn.college{kz_F2pbSUCr4BskErhXacY_APSE.0F0zITNywSMycjN4EzW}`

1.1.5. Using Your Own Frame

En este desafío se busca presentar que hagas uso de la pila que limpiaste para luego poder hacer comparaciones, con un tamaño específico de la pila y contar la cantidad de repetidos, buena forma de poder practicar lo aprendiendo anteriormente.

Flag Capturada

✓ CAPTURADA

`pwn.college{UP2DMXRmvENTsIQ66pSI2CeQvbg.0F01ATNywSMycjN4EzW}`

1.1.6. Environment Variables on the Stack

Justo después de la lista de argumentos(`argc`, `argv`), el núcleo coloca las variables de entorno. Al igual que `argv`, estas se almacenan en la pila como una matriz de punteros a cadenas, en la que cada cadena incluye el valor de la variable de la siguiente manera `HOME=/home/user/...`

En caso el programa sea llamado sin argumentos, y una única variable de entorno llamada `FLAG`, la estructura de la pila podría ser la siguiente:

Address	Contents	
+-----+		
<code>rsp + 0</code>	1	<--- <code>argc</code>
+-----+		
<code>rsp + 8</code>	<code>rsp + 128</code>	--+ <code>argv[0]</code> : pointer to the program name
+-----+		
<code>rsp + 16</code>	0	NULL (end of <code>argv</code>)
+-----+		
<code>rsp + 24</code>	<code>rsp + 200</code>	--++ <code>envp[0]</code> : pointer to the first env var
+-----+		
<code>rsp + 32</code>	0	NULL (end of <code>envp</code>)
+-----+		
+-----+		
Address	Contents	
+-----+		
<code>rsp + 128</code>	<code>"/tmp/..."</code>	<--+ the program name
+-----+		
...	...	
+-----+		
<code>rsp + 200</code>	<code>"FLAG=..."</code>	<-----+ the first env var: the <code>`FLAG`</code> variable
+-----+		

Tenemos que tener en cuenta 2 cosas:

1. Tanto argv y envp terminan en **NULL** al final de cada lista de punteros.
2. Las cadenas de envp tiene el formato **NOMBRE=VALOR**. Por lo tanto, envp[0] apunta a la cadena que comienza con el nombre de la primavera variable de entorno.

Flag Capturada

✓ CAPTURADA

flag

1.1.7. Aligning the Stack Through the Environment

Flag Capturada

✓ CAPTURADA

flag

1.1.8. Aligning the Stack Through GDB

Flag Capturada

✓ CAPTURADA

flag

1.1.9. Aligning the Stack Through GDB, Generalized

Flag Capturada

✓ CAPTURADA

flag